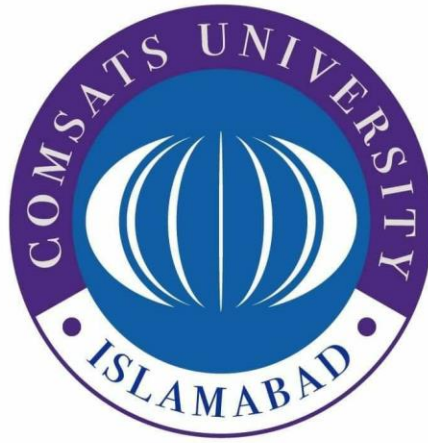




Department of Computer Science,
Attock Campus.

Program: BSSE

LAB Assignment 03

Information Security

Student Name	Syed Nazr-e-Abbas
Registration #	FA24-BSE-053

Instructor: Mam Ambreen Gull

Task 1: Basic Blockchain Implementation

Code:

```
import hashlib
import datetime

# Block Class
class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_string = str(self.index) + str(self.timestamp) + str(self.data) +
str(self.previous_hash)
        return hashlib.sha256(block_string.encode()).hexdigest()

# Blockchain Class
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, datetime.datetime.now(), "Pakistan Genesis Block", "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, data):
        previous_block = self.get_latest_block()

        new_block = Block(
            len(self.chain),
            datetime.datetime.now(),
            data,
            previous_block.hash
        )

        self.chain.append(new_block)

    def display_chain(self):
        for block in self.chain:
            print("\n-----")
            print("Index:", block.index)
            print("Timestamp:", block.timestamp)
            print("Data:", block.data)
            print("Previous Hash:", block.previous_hash)
            print("Hash:", block.hash)

# Create Blockchain
blockchain = Blockchain()

# Transactions
blockchain.add_block("Ali from Islamabad sent 5000 to Ahmed in Lahore")
```

```

blockchain.add_block("Abiha from Karachi paid 2500 to Hina in Rawalpindi")
blockchain.add_block("Abdullah from Peshawar sent 7000 to Bilal in Attock")
blockchain.add_block("Fatima from Faisalabad paid 1500 to Abbas in Attock")

# Display Blockchain
print("\nOriginal Blockchain:")
blockchain.display_chain()

# Tampering Demonstration
print("\nTampering with Block 2...\n")

blockchain.chain[2].data = "Hacked Transaction in Karachi"
blockchain.chain[2].hash = blockchain.chain[2].calculate_hash()

# Display After Tampering
print("\nBlockchain After Tampering:")
blockchain.display_chain()

```

Output:

```

PS C:\Users\Engineer Computers\Desktop\Information Security lab> & "C:/Users/Engineer Computers/A
on/Python314/python.exe" "C:/Users/Engineer Computers/Desktop/Information Security lab/A3_Task1.p

```

Original Blockchain:

```

-----
Index: 0
Timestamp: 2026-05-03 22:31:08.445005
Data: Pakistan Genesis Block
Previous Hash: 0
Hash: 2b5f1b690b99ba19f7de4c8e6f498d8d3f83213c3060773f95481c5aad5201fa

-----
Index: 1
Timestamp: 2026-05-03 22:31:08.445369
Data: Ali from Islamabad sent 5000 to Ahmed in Lahore
Previous Hash: 2b5f1b690b99ba19f7de4c8e6f498d8d3f83213c3060773f95481c5aad5201fa
Hash: aa2986d8720296edeec68642cb80bca9135f4ecfde344e2357d6550e26bc724b

-----
Index: 2
Timestamp: 2026-05-03 22:31:08.445392
Data: Abiha from Karachi paid 2500 to Hina in Rawalpindi
Previous Hash: aa2986d8720296edeec68642cb80bca9135f4ecfde344e2357d6550e26bc724b
Hash: 20e785e9db8748754e0f6a1bd5a5773587b653a9964c4bdc8831ddf5985236cf

-----
Index: 3
Timestamp: 2026-05-03 22:31:08.445401
Data: Abdullah from Peshawar sent 7000 to Bilal in Attock
Previous Hash: 20e785e9db8748754e0f6a1bd5a5773587b653a9964c4bdc8831ddf5985236cf
Hash: 6e5b60b19979e5deadd72e0d546b0cd5d4c202389dec87d17354e1f10a7fbad2

-----
Index: 4
Timestamp: 2026-05-03 22:31:08.445411
Data: Fatima from Faisalabad paid 1500 to Abbas in Attock
Previous Hash: 6e5b60b19979e5deadd72e0d546b0cd5d4c202389dec87d17354e1f10a7fbad2
Hash: 4a10228bc32a7611bb4a6940efc3b9fe6b8fa1393126b0e772519b6ba27fb689

```

Tampering with Block 2...

Blockchain After Tampering:

```

-----
Index: 0
Timestamp: 2026-05-03 22:31:08.445005
Data: Pakistan Genesis Block
Previous Hash: 0
Hash: 2b5f1b690b99ba19f7de4c8e6f498d8d3f83213c3060773f95481c5aad5201fa

-----
Index: 1
Timestamp: 2026-05-03 22:31:08.445369
Data: Ali from Islamabad sent 5000 to Ahmed in Lahore
Previous Hash: 2b5f1b690b99ba19f7de4c8e6f498d8d3f83213c3060773f95481c5aad5201fa
Hash: aa2986d8720296edeec68642cb80bca9135f4ecfde344e2357d6550e26bc724b

-----
Index: 2
Timestamp: 2026-05-03 22:31:08.445392
Data: Hacked Transaction in Karachi
Previous Hash: aa2986d8720296edeec68642cb80bca9135f4ecfde344e2357d6550e26bc724b
Hash: fdeec42938328505a86ae1e11833441b1436e38d5a436b0a08f4f7c836ca4a7b

-----
Index: 3
Timestamp: 2026-05-03 22:31:08.445401
Data: Abdullah from Peshawar sent 7000 to Bilal in Attock
Previous Hash: 20e785e9db8748754e0f6a1bd5a5773587b653a9964c4bdc8831ddf5985236cf
Hash: 6e5b60b19979e5deadd72e0d546b0cd5d4c202389dec87d17354e1f10a7fbad2

-----
Index: 4
Timestamp: 2026-05-03 22:31:08.445411
Data: Fatima from Faisalabad paid 1500 to Abbas in Attock
Previous Hash: 6e5b60b19979e5deadd72e0d546b0cd5d4c202389dec87d17354e1f10a7fbad2
Hash: 4a10228bc32a7611bb4a6940efc3b9fe6b8fa1393126b0e772519b6ba27fb689
PS C:\Users\Engineer Computers\Desktop\Information Security lab>

```

Explanation of Blockchain Code

1. Import Libraries

```
import hashlib  
import datetime
```

- hashlib is used to generate SHA256 hashes.
- datetime is used to timestamp each block.

2. Block Class

```
class Block:
```

This class represents one block in the blockchain.

Each block contains:

- Index
- Timestamp
- Data
- Previous Hash
- Current Hash

3. Constructor

```
def __init__(self, index, timestamp, data, previous_hash):
```

This initializes block attributes.

4. Hash Calculation

```
def calculate_hash(self):
```

This function:

- Combines block contents
- Converts them into bytes
- Uses SHA256 to create a unique hash

SHA256 provides:

- Fixed-length output
- Collision resistance
- Integrity verification

5. Blockchain Class

```
class Blockchain:
```

This class manages all blocks.

6. Genesis Block

```
self.chain = [self.create_genesis_block()]
```

The Genesis Block is the first block.

7. Add Block

```
def add_block(self, data):
```

This function:

- Gets latest block
- Creates a new block
- Uses previous block hash
- Adds block to chain

8. Tampering Demonstration

When block data changes:

```
blockchain.chain[2].data = "Hacked Transaction in Karachi"
```

The block hash changes.

Since the next block stores the previous hash, the chain becomes invalid.

Blockchain Security Analysis

Strengths

1. **Immutability**
 - Changing one block changes its hash.
 - Chain integrity breaks.
2. **Integrity**
 - SHA256 ensures data authenticity.
3. **Linking Through Hashes**
 - Blocks depend on previous hashes.

Weaknesses

1. No proof-of-work
2. No digital signatures
3. No distributed nodes
4. No consensus mechanism

Task 2: Elliptic Curve Cryptography (ECC)

Code:

```
# ECC Parameters
a = 2
b = 2
p = 17
# Base Point
G = (5, 1)
# Modular Inverse Function
def mod_inverse(k, p):
    for i in range(1, p):
        if (k * i) % p == 1:
            return i
    return None
# Point Addition
def point_add(P, Q):
    if P == Q:
        return point_double(P)
    x1, y1 = P
    x2, y2 = Q
    slope = ((y2 - y1) * mod_inverse(x2 - x1, p)) % p
    x3 = (slope**2 - x1 - x2) % p
    y3 = (slope * (x1 - x3) - y1) % p
    return (x3, y3)
# Point Doubling
def point_double(P):
    x1, y1 = P
    slope = ((3 * x1**2 + a) * mod_inverse(2 * y1, p)) % p
    x3 = (slope**2 - 2 * x1) % p
    y3 = (slope * (x1 - x3) - y1) % p
    return (x3, y3)
# Scalar Multiplication
def scalar_multiply(k, P):
    result = P
    for _ in range(k - 1):
        result = point_add(result, P)
    return result
# Key Generation
private_key = 4
```

```

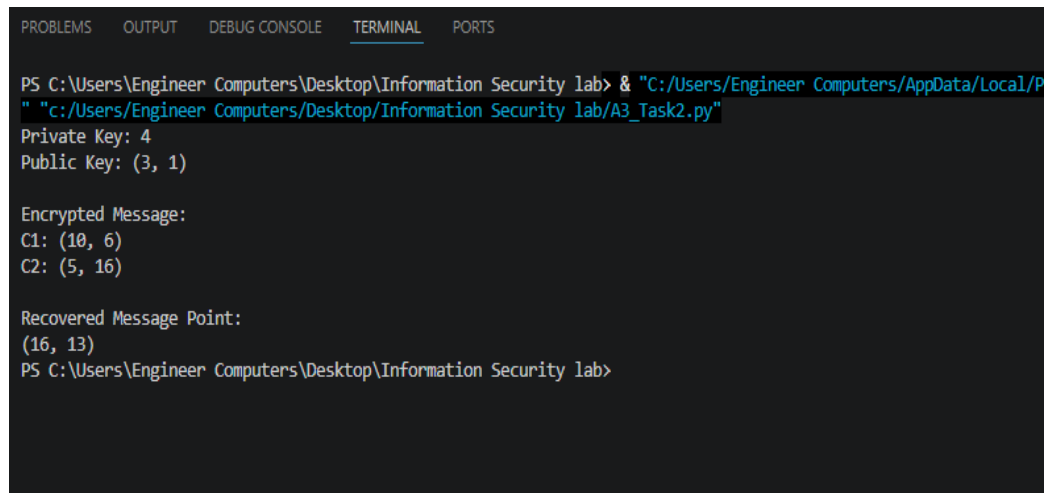
public_key = scalar_multiply(private_key, G)
print("Private Key:", private_key)
print("Public Key:", public_key)
#Message Example
message = 6
M = scalar_multiply(message, G)
# Encryption
k = 3
C1 = scalar_multiply(k, G)
C2 = point_add(M, scalar_multiply(k, public_key))
print("\nEncrypted Message:")
print("C1:", C1)
print("C2:", C2)
# Decryption
shared_secret = scalar_multiply(private_key, C1)

neg_shared_secret = (
    shared_secret[0],
    (-shared_secret[1]) % p
)
recovered_message = point_add(C2, neg_shared_secret)

print("\nRecovered Message Point:")
print(recovered_message)

```

Output:



```

PS C:\Users\Engineer Computers\Desktop\Information Security lab> & "C:/Users/Engineer Computers/AppData/Local/Programs/Python/Python39-64/Python.exe" "c:/Users/Engineer Computers/Desktop/Information Security lab/A3_Task2.py"
Private Key: 4
Public Key: (3, 1)

Encrypted Message:
C1: (10, 6)
C2: (5, 16)

Recovered Message Point:
(16, 13)
PS C:\Users\Engineer Computers\Desktop\Information Security lab>

```

Explanation of ECC Code

1. ECC Curve

$$y^2 = x^3 + ax + b \pmod{p}$$

Curve Parameters:

- $a = 2$
- $b = 2$
- $p = 17$

2. Point Addition

Formula:

- Compute slope
- Compute x_3
- Compute y_3

3. Point Doubling

```
slope = ((3 * x1**2 + a) * mod_inverse(2 * y1, p)) % p
```

Used when adding a point to itself.

4. Scalar Multiplication

```
for _ in range(k - 1):
```

5. Key Generation

Private key:

```
private_key = 4
public_key = scalar_multiply(private_key, G)
```

6. EC-ElGamal Encryption

Encryption Formula:

```
C1 = scalar_multiply(k, G)
C2 = point_add(M, scalar_multiply(k, public_key))
```

7. Decryption

Receiver computes:

```
shared_secret = scalar_multiply(private_key, C1)

neg_shared_secret = (
    shared_secret[0],
    (-shared_secret[1]) % p
)
```

ECC Security Analysis

Strengths

1. Small key size
2. High security level
3. Efficient computations
4. Widely used in cryptocurrencies

Weaknesses

1. Complex implementation
2. Vulnerable if weak curve chosen
3. Small educational curves are not secure